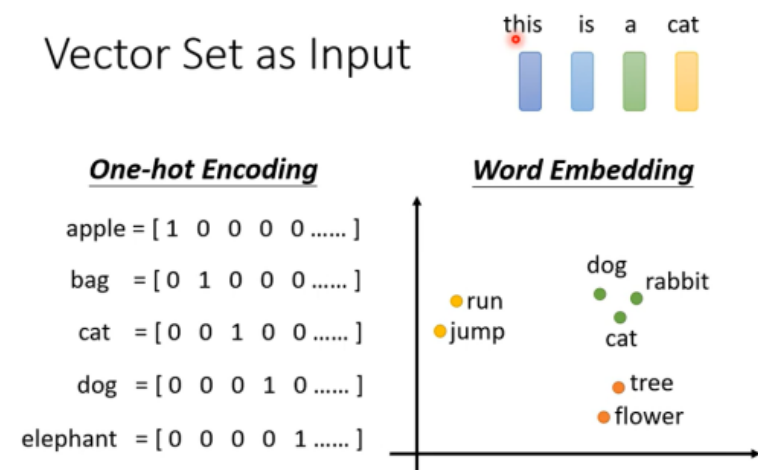


Self-attention(自注意力机制)

当输入是一排向量，且向量数量会改变时

input

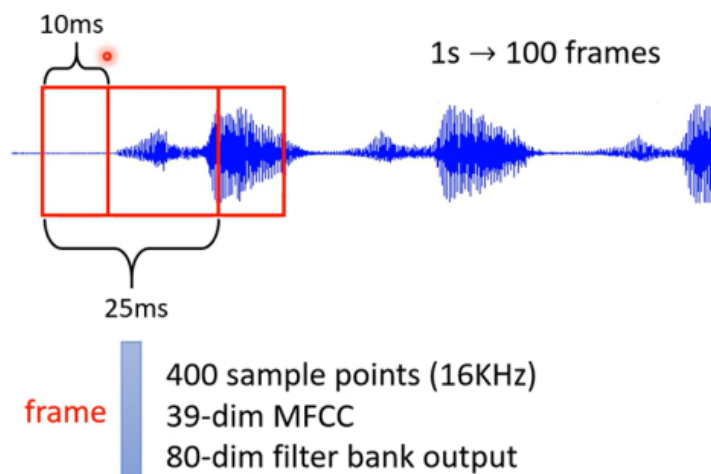
1. 文字处理



不能用独热编码：

1. 词汇表有几个词，就会有几个one-hot向量
2. one-hot Encoding假设各个单词之间毫无联系，无法捕捉单词间联系
常用：Word Embedding(词嵌入)，将每个单词表示为一个向量，联系近的距离近

2. 语音处理



一段语音，每一小段(25ms)看作一个向量，每次向后移动10ms生成一个新向量

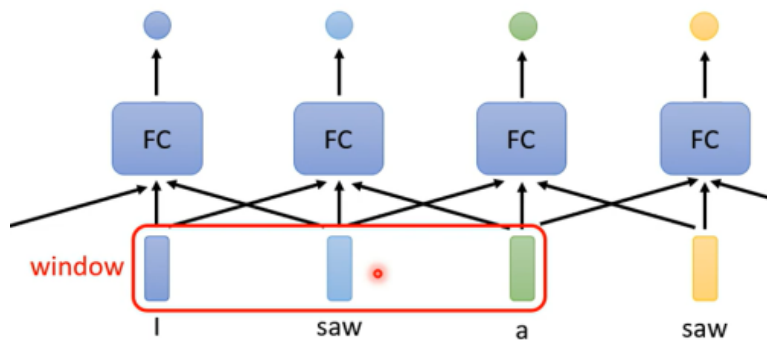
3. Graph(社交网络、分子)

每个点的资讯看作一个向量

output

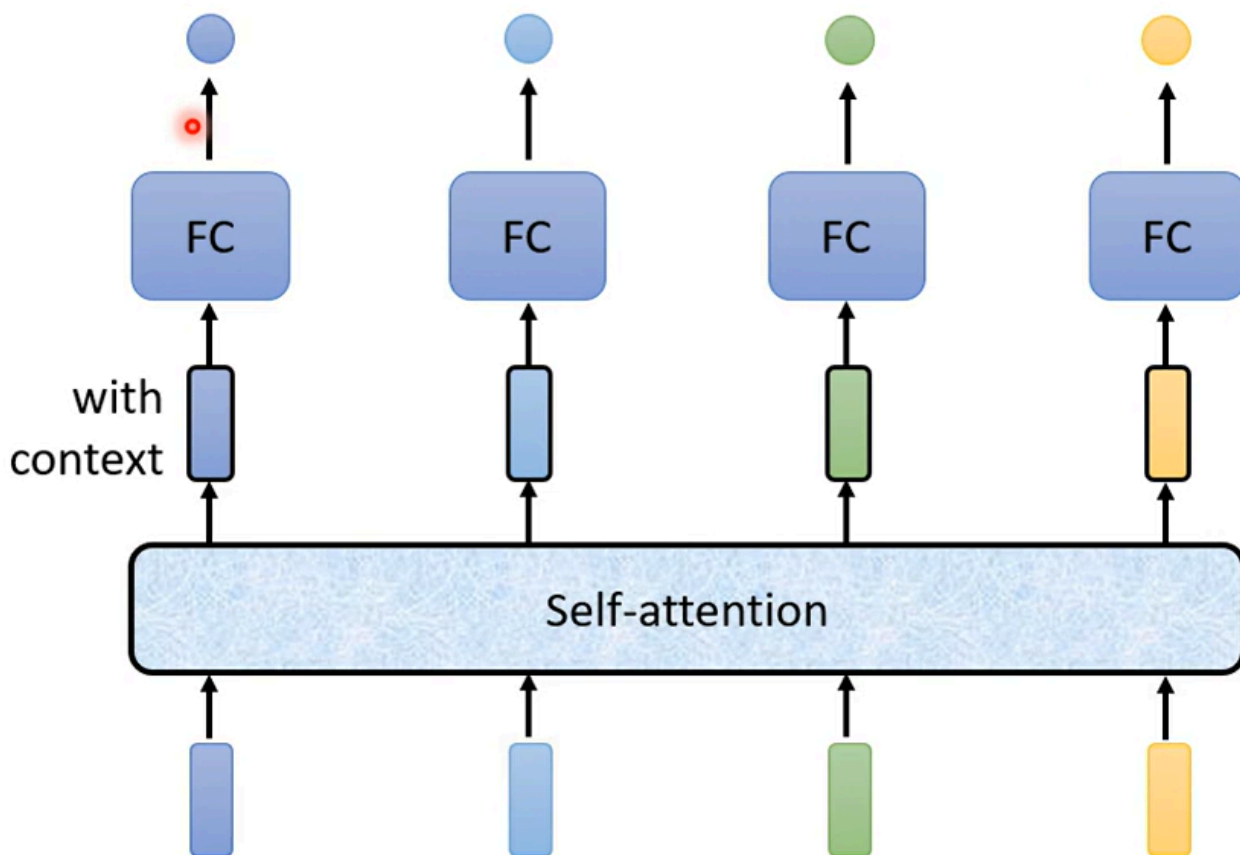
1. each vec has a lable
2. the whole sequence has a lable
3. Model decides the number of labels itself(seq2seq)

第一种情况



在对文字处理时，只用fully connect layer时window大小是固定的，无法很好的应对长度有大有小的情况
∴用self-attention

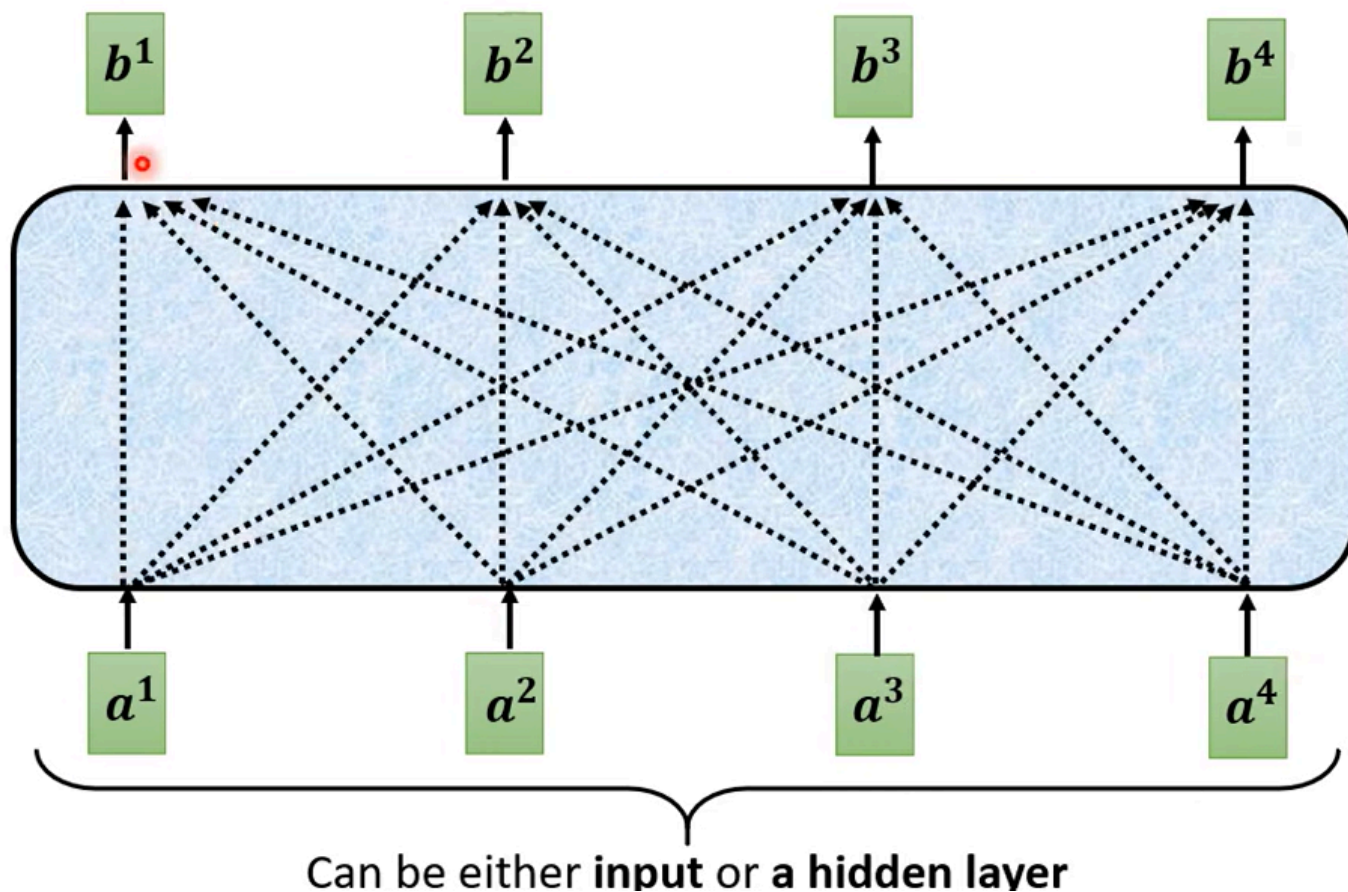
Self-attention



self-attention会记住一整个seq的资讯，读完后在输出单个vec。输出的vec数量跟输入的相同

self-attention也可以和fully connect交替使用

Self-attention

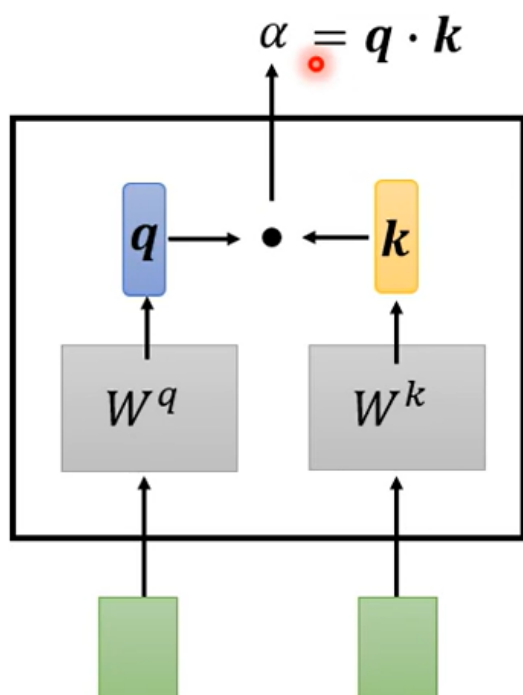


每个输出都是考虑了所有输入才生成的

1. 计算 $\alpha_{1,i}$ (attention score):

根据 a^1 找到跟 a^1 相关的向量，用 α 表示两个向量的关联程度

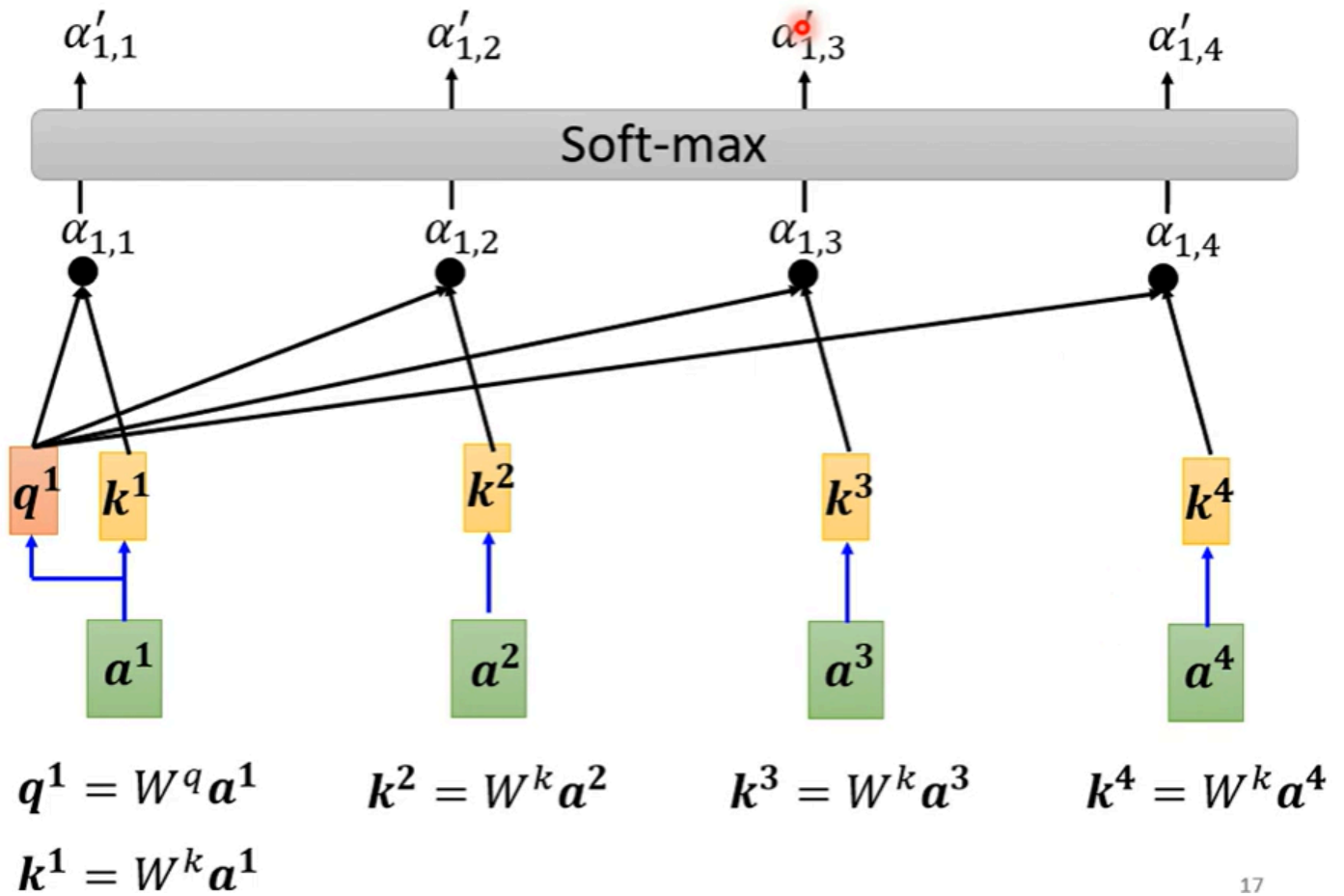
Dot-product(点乘):



将两个向量乘不同矩阵得到 q 向量和 k 向量， $\alpha = q \cdot k$

Self-attention

$$\alpha'_{1,i} = \exp(\alpha_{1,i}) / \sum_j \exp(\alpha_{1,j})$$



17

q^i : 查询向量

k^i : 键向量

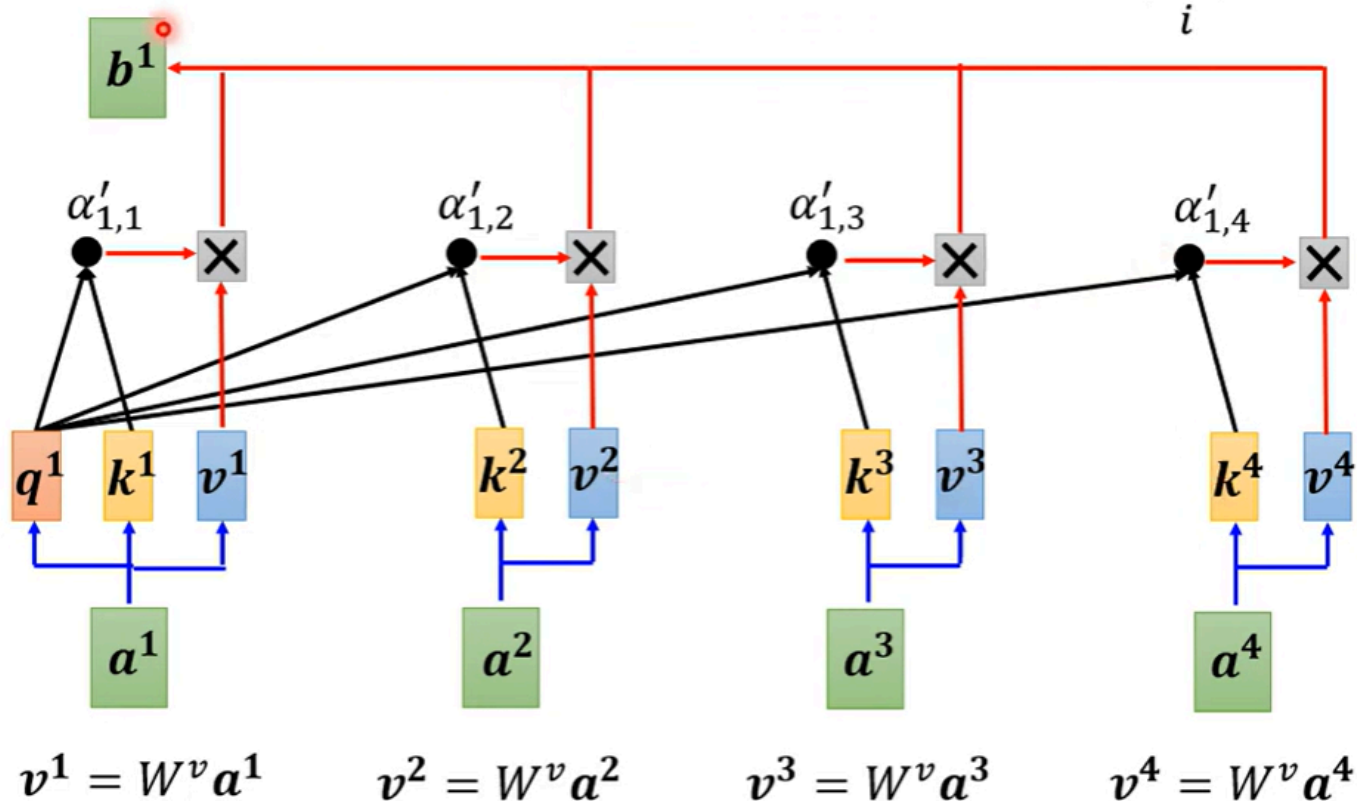
将查询向量分别和其他键向量做点乘后得到 α 后，对 α 做归一化(softmax、ReLU等都行)

2. 求 b^1

Self-attention

Extract information based on attention scores

$$b^1 = \sum_i \alpha'_{1,i} v^i$$



将每个 a^i 乘 W^v 得到 v^i ，与 α' 相乘再求和得到 b^1
 $\alpha'_{1,i}$ 越大， b^1 中该 v^i 占比就越大

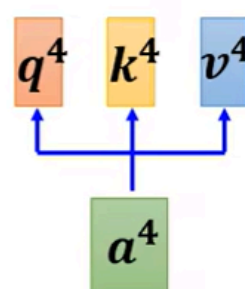
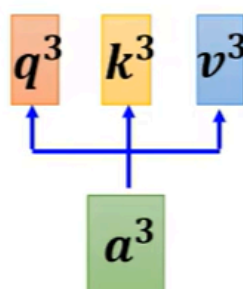
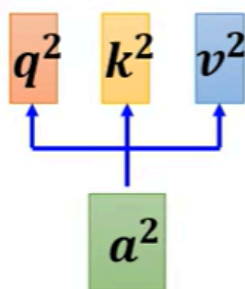
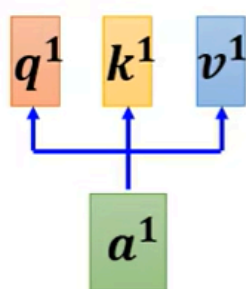
3. 将对每个 a^i 的操作结合一下~

Self-attention

$$q^i = W^q a^i \quad \begin{matrix} q^1 & q^2 & q^3 & q^4 \\ Q \end{matrix} = \begin{matrix} W^q & a^1 & a^2 & a^3 & a^4 \\ & I \end{matrix}$$

$$k^i = W^k a^i \quad \begin{matrix} k^1 & k^2 & k^3 & k^4 \\ K \end{matrix} = \begin{matrix} W^k & a^1 & a^2 & a^3 & a^4 \\ & I \end{matrix}$$

$$v^i = W^v a^i \quad \begin{matrix} v^1 & v^2 & v^3 & v^4 \\ V \end{matrix} = \begin{matrix} W^v & a^1 & a^2 & a^3 & a^4 \\ & I \end{matrix}$$



查询矩阵(Query): $Q = W^q I$

键矩阵(Key): $K = W^k I$

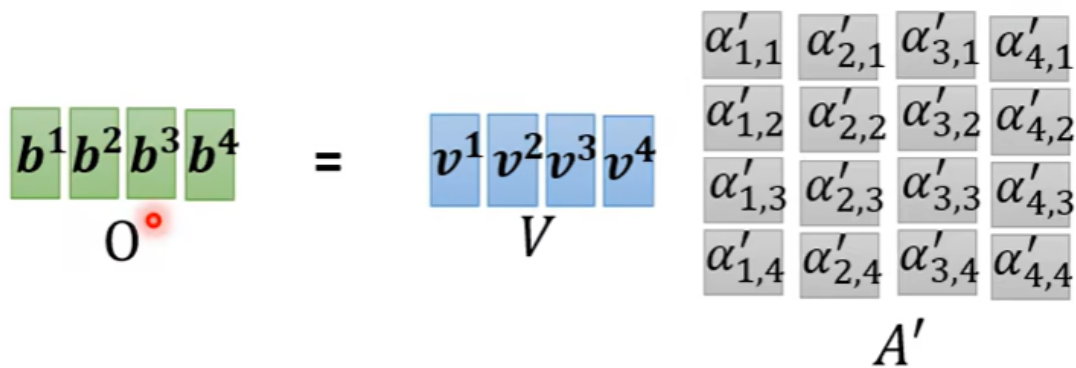
值矩阵(Value): $V = W^v I$

$$\begin{matrix} \alpha'_{1,1} & \alpha'_{2,1} & \alpha'_{3,1} & \alpha'_{4,1} \\ \alpha'_{1,2} & \alpha'_{2,2} & \alpha'_{3,2} & \alpha'_{4,2} \\ \alpha'_{1,3} & \alpha'_{2,3} & \alpha'_{3,3} & \alpha'_{4,3} \\ \alpha'_{1,4} & \alpha'_{2,4} & \alpha'_{3,4} & \alpha'_{4,4} \end{matrix} \xleftarrow{\text{softmax}} \begin{matrix} \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4} \end{matrix} = \begin{matrix} k^1 \\ k^2 \\ k^3 \\ k^4 \\ K^T \end{matrix} \begin{matrix} q^1 & q^2 & q^3 & q^4 \\ Q \end{matrix}$$

注意力矩阵(Attention):

$$A = K^T Q$$

$$A' = \text{softmax}(A)$$



(注：图片中的 $\hat{\alpha}$ 其实是 α')

输出矩阵O：

$$O = VA'$$

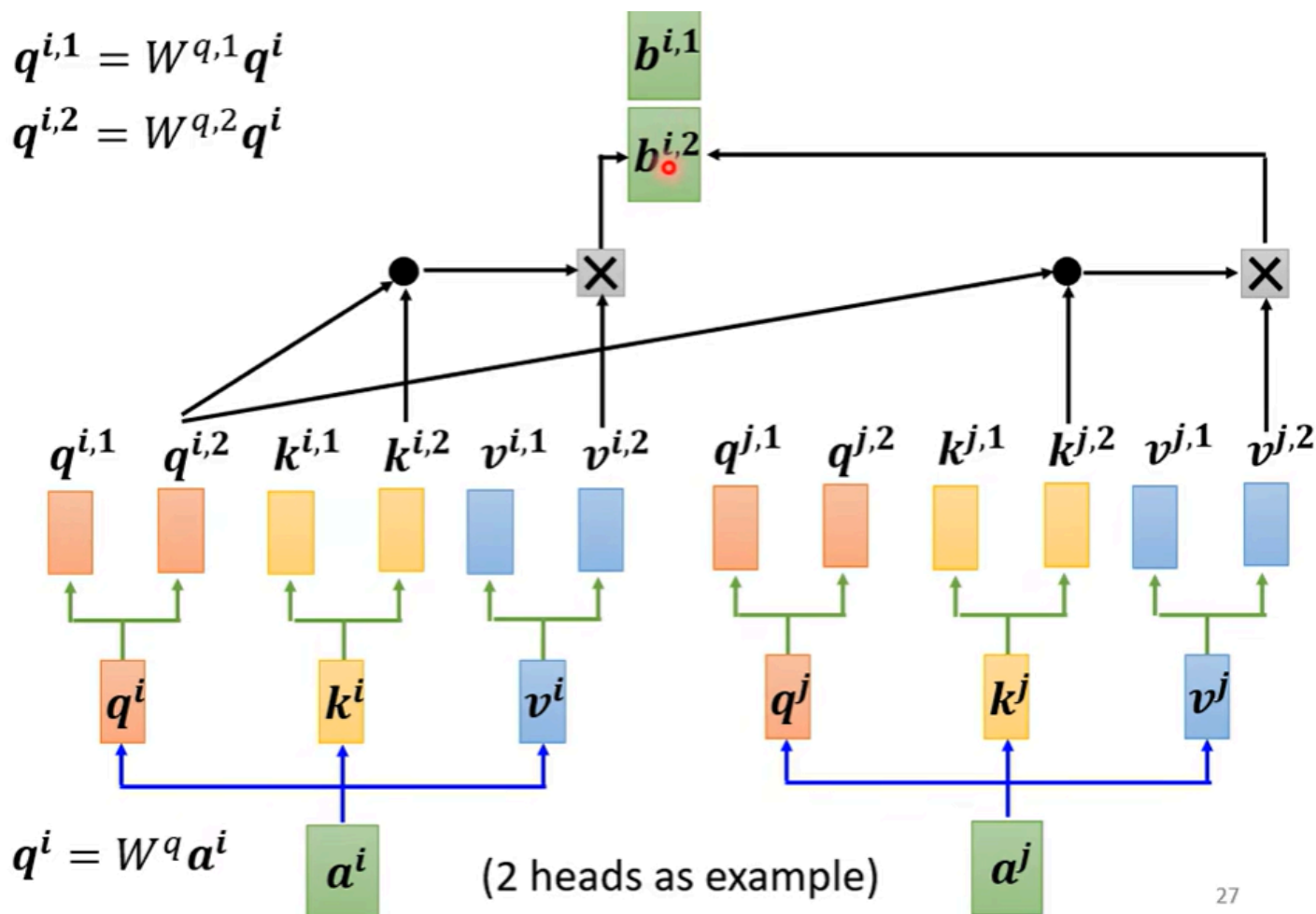
综上，只有 W^q, W^k, W^v 是需要学习的参数

Multi-head-Self-attention

在self-attention中，使用 q 去找相关的 k ，但相关有多种形式，所以需要多个 q ，不同 q 负责不同的相关性

$$q^{i,1} = W^{q,1} q^i$$

$$q^{i,2} = W^{q,2} q^i$$



$$b^i = W^o \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

当用两个 q 时，将每个 q 分别与该相关性对应的 k 相乘得到该相关性的output
然后将不同相关性的output连接起来transform一下作为下一层的input

Positional Encoding

在做self-attention时，机器不知道各个词汇之间的位置资讯，但是位置资讯有可能很重要，所以考虑把位置资讯加入训练

为每个位置设定一个独特的位置向量 e^i ， e^i 的设置方法有待研究

self-attention for speech

由于语音辨识一次只能看10ms，所以当语音很长时，attention matrix会很大，但是要判断某个位置有什么样的东西，只需要看周围小范围的资讯就好了

所以考虑使用Truncated Self-attention，即每次只选一小范围的片段做辨识，范围由人为设定

self-attention for image

每个像素看作是一个三维向量，则一张图片也是一个vector set

VS CNN

self-attention的func set比CNN更大，使用更灵活，但更复杂，需要data更多

VS RNN

1. self-attention更容易考虑两个距离较远的input的关系
2. self-attention的输出是平行的

self-attention for graph

在计算attention matrix时，可以只计算有edge相连的node